

Sugestões de Desenvolvimento

1. Convenção de Commits

Cada commit deve ter um prefixo que indica a ação realizada.

1.1. Tipos de Commits

A tabela abaixo apresenta os tipos de commits padronizados e exemplos práticos:

Prefixo	Descrição	Exemplo
<code>create</code>	Criar uma nova funcionalidade ou módulo	<code>create: endpoint de autenticação do backend</code>
<code>add</code>	Adicionar a um recurso existente sem grandes modificações	<code>add: DTO para usuários</code>
<code>update</code>	Atualizar um recurso existente sem grandes modificações	<code>update: refatoração no serviço de autenticação</code>
<code>fix</code>	Corrigir algum bug	<code>fix: correção no middleware de autenticação</code>
<code>delete</code>	Remover um recurso, código ou arquivo	<code>delete: endpoint de usuários inativos</code>
<code>refactor</code>	Melhorar o código sem alterar a funcionalidade	<code>refactor: otimização na query de produtos</code>
<code>docs</code>	Atualizar documentação	<code>docs: atualização do README.md</code>
<code>test</code>	Adicionar ou corrigir testes	<code>test: cobertura para o serviço de pagamentos</code>
<code>merge</code>	Realizar merge entre branches	<code>merge: feature/cadastro into main</code>
<code>hotfix</code>	Correção rápida de um erro crítico	<code>hotfix: correção na API de checkout</code>

2. Estratégia de Branching para Tarefas

Como as tarefas são passadas por "task", o fluxo pode seguir este modelo:

2.1. Fluxo de Desenvolvimento

1. Criar uma branch específica para cada tarefa (task) utilizando um padrão definido:
 - `feature/nome-da-feature` (Para novas funcionalidades)
 - `bugfix/nome-do-bug` (Para correção de bugs)
 - `hotfix/nome-do-hotfix` (Para correção rápida de erros críticos)
2. Implementar e testar as alterações na branch da task.
3. Realizar commits seguindo a convenção padronizada.
4. Criar um Pull Request para revisão antes do merge para a branch principal.
5. Após a aprovação e testes finais, realizar o merge para a branch principal.

3. Estratégias para Comunicação entre Backend e Frontend

Para melhorar a integração entre backend e frontend, recomenda-se:

3.1. Uso de DTOs (Data Transfer Objects)

Os **DTOs** são essenciais para definir quais dados serão trocados entre backend e frontend, garantindo maior consistência na comunicação.

Exemplo em FastAPI:

```
from pydantic import BaseModel, EmailStr
from typing import Optional

# DTO para criação de usuário
class UserCreatedDTO(BaseModel):
    name: str
    email: EmailStr
    password: str

# DTO para resposta do usuário (evita expor a senha)
class UserResponseDTO(BaseModel):
    id: int
    name: str
    email: EmailStr
```

Exemplo em Next.js:

```
// DTO para criação de usuário
export interface UserCreatedDTO {
  name: string;
  email: string;
  password: string;
}

// DTO para resposta da API
export interface UserResponseDTO {
  id: number;
  name: string;
  email: string;
}
```

3.2. API Documentation (Swagger)

Utilizar ferramentas como **Swagger** para documentar as APIs, permitindo que o frontend visualize facilmente os endpoints disponíveis e suas respectivas estruturas de resposta.

3.3. Padrões de Respostas da API

Para evitar ambiguidades, manter um padrão claro nas respostas da API. **Exemplo de resposta padronizada:**

```
{
  "status": "success",
  "data": {
    "id": 1,
    "name": "João Pedro",
    "email": "joao@email.com"
  },
  "message": "Usuário cadastrado com sucesso"
}
```

3.4. Controle de Versionamento da API

Para evitar quebras de compatibilidade, utilizar versionamento nas APIs. **Exemplo:**

- `/api/v1/usuarios`
- `/api/v2/usuarios`

4. Testes

Essenciais para garantir a qualidade do código e evitar falhas em produção. Os testes devem abranger diferentes camadas do sistema, validando a funcionalidade de cada componente. No desenvolvimento das tasks focaremos em dois tipos, utilizando o pytest:

Testes Unitários:

- Validam partes isoladas do código, como funções e serviços.
- Exemplo: Testar um serviço de autenticação no backend.

Testes de Integração:

- Avaliam a interação entre diferentes módulos do sistema.
- Exemplo: Verificar se o backend retorna os dados corretamente para o frontend.

Ao final do software, implementar o seguinte teste:

Testes End-to-End (E2E):

- Simulam o uso real do sistema do início ao fim.
- Exemplo: Criar um usuário e testar todo o fluxo de login.

5. Sugestões Gerais

- Evitar inserir autenticação diretamente nos recursos, criando módulos próprios de autenticação separados.